

Trabalho 2 - SGBD - Índices Hashing

Javam Machado

Abril de 2026

1 Aspectos Gerais

Um banco de dados armazenado em disco possui uma relação que armazena trechos do livro Dom Casmurro. O objetivo deste trabalho é implementar um índice **Hash Extensível** para otimizar a recuperação dessas frases com base em seus identificadores únicos.

Cada equipe implementará um índice hash extensível. O índice deve considerar que os dados estão armazenados em um arquivo de disco (**bd-trab2 - dataset.csv**), no qual cada linha (registro) representa uma página da tabela do banco de dados, contendo trechos do livro Dom Casmurro. A equipe deve implementar as operações de busca por igualdade, recuperando do arquivo de dados o registro de interesse, inserção e remoção de entradas de dados no índice.

Livro
int LinhaNum (PK) string LinhaTexto

Figure 1: Relação a ser indexada.

2 Detalhamento

Neste trabalho, um banco de dados mapeado em **disco** terá uma única tabela chamada “Livro”, com tuplas contendo os atributos “LinhaNum” (identificador único) e “LinhaTexto” (conteúdo da linha) – Figura 1. Cada registro corresponde a um trecho do livro e, conceitualmente, cada página contém apenas um único registro. A tabela “Livro” está armazenada em um arquivo em disco chamado “ArquivoTexto” (bd-trab2-dataset.csv), no qual cada linha representa uma tupla.

Além disso, o banco possui um índice hash extensível sobre o atributo “LinhaNum”, implementado por meio de dois arquivos: um para o Diretório (vetor de ponteiros para *buckets*) e outro para os Buckets, que armazenam as entradas de dados do índice contendo o valor de “LinhaNum”. Observe que, neste trabalho, “LinhaNum” atua tanto como chave de busca quanto como identificador do registro (RID), permitindo localizar diretamente a tupla no arquivo de dados. Seu banco de dados será portanto formado pelos arquivos ArquivoTexto, Diretório e os Buckets.

A Figura 2 contém a representação do banco de dados. Nela, pode-se identificar o “Arquivo de texto”, que corresponde ao conjunto de dados (**bancodedados.csv**) utilizado no primeiro trabalho. Para facilitar a implementação, cada registro desse arquivo representa uma página da tabela no banco de dados. Por exemplo, se o arquivo possuir 4 páginas, isso será representado por meio de 4 registros no Arquivo de texto.

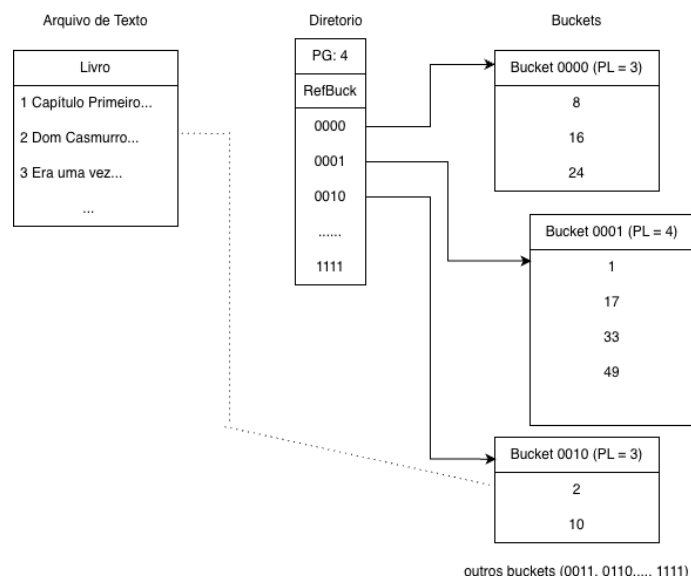


Figure 2: Representação do Banco

Na Figura 2 você pode identificar a estrutura de um diretório. Ele contém uma profundidade global – variável PG iniciada com valor 4 – e uma sequência de referências para os *buckets*. A implementação do trabalho deve usar a função hash que recupera os PG bits menos significativos do identificador da linha (campo LinhaNum). Cada entrada do Diretorio contém uma referência para o seu *bucket* correspondente.

Ainda na Figura 2 você pode identificar a área de páginas primárias do índice hash. Cada *bucket* será representado por um arquivo, em que cada linha corresponde a uma entrada de dados do *bucket*, cujo valor é o identificador da linha para permitir sua recuperação direta. Um *bucket* terá, no máximo, 5 entradas de dados; ou seja, cada arquivo correspondente a um *bucket* armazena, no máximo, 5 registros. Cada *bucket* contém uma variável que representa sua profundidade local.

Observações importantes:

- **Somente uma única página de dados deve estar em memória por vez!** Ou seja, não é permitido manter todos os dados em memória simultaneamente. Única exceção para essa restrição: diretório do hash.
- Deverá existir um arquivo **Main** que será responsável por ler um arquivo de texto com as informações de entrada, bem como gerar um arquivo de texto com as informações de saída. Mais informações a seguir.

O arquivo **Main** será o responsável por realizar chamadas as funcionalidades implementadas. Para tanto, um único arquivo de entrada **in.txt** será fornecido. Nesse arquivo de entrada, estarão as operações de índice que devem ser realizadas, sendo que cada linha desse arquivo é composta por uma das alternativas seguintes:

INC: x
 REM: x
 BUS=: x

De maneira intuitiva, INC, REM, BUSC=, representam, respectivamente, as operações de inclusão, remoção e busca por igualdade no índice hash. Ademais, em todas as operações, **x** representa o inteiro a ser utilizado na respectiva operação.

A primeira linha do arquivo de entrada, antes das operações em si, irá indicar a profundidade global do hash extensível. A sintaxe dessa linha é a seguinte:

PG/<profundidade global inicial>

Por fim, um arquivo de saída nomeado de **out.txt** deve ser gerado. A linha inicial do arquivo deve ser igual à primeira linha do arquivo de entrada e, após cada operação do arquivo de entrada ser realizada, deve ser incluída uma linha no arquivo de saída da seguinte forma:

```
INC:x/<profundidade global>,<profundidade local>
REM:x/<qtd de tuplas removidas>,<profundidade global>,<profundidade local>
BUS:x/<quantidade de tuplas selecionadas>
```

No arquivo do índice em hash extensível, deve-se adicionar a seguinte linha sempre logo após a inclusão de um elemento que duplica o diretório: DUP_DIR:/<profundidade global>,<profundidade local>(antes de INC:x). Além disso, a última linha do arquivo de saída deve ser P:/<profundidade global final>.

3 Requisitos

- Linguagem de programação: C/C++;
- Trabalho em equipe de no máximo 2 alunos;
- Comentar o código implementado para facilitar a sua verificação;
- Escreva um *Readme* descrevendo o projeto e como executá-lo;
- Comprimir o seu projeto (código-fonte e executável), e o *Readme* em um único pacote e submeter via **Classroom**. O nome do projeto comprimido deve conter o nome da dupla (*<Nome_Aluno_1>-<Nome_Aluno_2>.zip*);

4 Avaliação

Os trabalhos entregues serão avaliados pelos seguintes parâmetros:

- Garantir que somente uma única página de dados está em memória por vez;
- Construção correta dos buckets;
- Gestão correta do diretório do índice hash;
- Resposta claras e corretas à arguição quando da entrega;
- Organização e clareza do código, comentários e *Readme*;

5 Entrega

Data da entrega: 2a feira - 27 de abril de 2026 até 10h com apresentação e arguição no LEC/DC no mesmo dia, no horário da aula. O código do trabalho deve ser enviado no **classroom** até o final do horário da entrega. Envios em atraso serão penalizados. Quaisquer dúvidas acerca do trabalho podem ser enviadas aos monitores: Francisco das Chagas e Josué Hinrichs via **classroom**.